

PREUVES FORMELLES : fiches de révision

TP 1

Implication : La proposition $[P \rightarrow Q]$ signifie « si $[P]$, alors $[Q]$ » ou, plus précisément, « je sais contruire une preuve de Q à partir d'une preuve de P ».

• **Intros P :** soit P une proposition quelconque.

- Pour montrer une implication on suppose que ce qui est à gauche est prouvé. On doit prouver ce qui est à droite avec cette hypothèse supplémentaire.
- `[imp_refl]` est le nom que nous avons choisi pour ce théorème.
- La preuve se trouve entre les mots-clés `[Proof.]` et `[Qed.]`
- Les éléments de la preuve `[intros]` et `[assumption]` sont des `_tactiques_` de preuve.
- Lorsque l'évaluation par `coq` atteint `[Qed.]`, `coq` vérifie que la preuve est correcte et dans le cas contraire affiche un message d'erreur.

L'implication n'est pas associative :

- $[(P \rightarrow Q) \rightarrow R]$ n'est **pas** équivalente à $[P \rightarrow (Q \rightarrow R)]$, ces deux propositions ont des sens très différents.
- Quand on ne met pas de parenthèses dans une expression avec $[->]$, c'est comme s'il y en avait à droite : **$[P \rightarrow Q \rightarrow R]$ est la même proposition que $[P \rightarrow (Q \rightarrow R)]$.**
- En fait, plutôt que de penser à $[P \rightarrow Q \rightarrow R]$ comme $(P \text{ implique que } (Q \text{ implique } R))$, il vaut mieux y voir « **si P et Q , alors R** », ou encore, « **si j'ai une preuve de P et une preuve de Q , alors je sais contruire une preuve de R** ».

• **Apply :** une proposition est vrai par hypothèse.

```
(* Soient P et Q deux propositions quelconques. *)
  intros P Q.
(* Supposons que P est prouvée ... *)
  intros HP.
(* ... et que (P -> Q) est prouvée. *)
  intros HPimpQ.
(* Comme (P -> Q), pour montrer Q _il suffit_ de montrer P. *)
  apply HPimpQ.
(* Or, P est vraie par hypothèse. *)
  assumption.
```

Si la formule à prouver est $[Q]$ et qu'une hypothèse $[H]$ a la forme $[P \rightarrow Q]$, alors l'utilisation de `[apply H.]` entraîne le changement du but en $[P]$. Ceci est conforme à la règle appelée syllogisme ou `_modus ponens_` qui dit que de $[P \rightarrow Q]$ et $[P]$, on peut déduire $[Q]$.

- Socrate est un homme. $([P])$
- Tous les hommes sont mortels. $([P \rightarrow Q])$
- Donc Socrate est mortel. $([Q])$

*)

(* On peut (mais on n'est pas obligé) utiliser les caractères - + et * pour mettre en évidence le fait qu'on travaille sur un sous-but *) : -

**** ** Conjonction $[\wedge]$ *)**

(** Le "et logique" aussi appelé opérateur de conjonction est noté en `Coq` $[\wedge]$.
Si $[P]$ et $[Q]$ sont deux propositions, alors $[P \wedge Q]$ est une proposition dont

une preuve est à la fois une preuve de $[P]$ et une preuve de $[Q]$.
*)

(** *** Utiliser une conjonction : `[destruct]` *)

(** Pour décomposer une preuve de $[P \wedge Q]$ en une preuve de $[P]$ et une preuve de $[Q]$, on utilise la tactique `[destruct]`. *)

(** remarque: au lieu d'écrire "intros HPQ" suivie de "destruct HPQ as [HP HQ]" il est possible d'écrire simplement "intros [HP HQ]".
C'est un cas particulier de ce qu'on appelle "intro pattern" en coq.
À n'utiliser que si vous avez bien compris `[destruct]`. La preuve de `proj1` peut donc s'écrire simplement : *)

(** *** Prouver une conjonction : `[split]` *)

(** Prouver $[P \wedge Q]$ se fait en deux étapes : il faut une preuve de P et une preuve de Q . Ceci se fait avec la tactique `[split]`. *)

Theorem conj : forall P Q : Prop, P -> Q -> P ∧ Q.
Proof.
(* Soient P et Q deux propositions quelconques. *)
intros P Q.
(* Supposons :
(HP) : P
(HQ) : Q. *)
intros HP HQ.
(* Pour prouver $P \wedge Q$ on doit prouver P puis Q. *)
split.
- (* Preuve de P : *)
(* P est prouvée par hypothèse. *)
assumption.
- (* Preuve de Q : *)
(* Q est prouvée par hypothèse. *)
assumption.
Qed.

(** Remarque : comme pour `[->]` l'opérateur `[∧]` est "associatif à droite" en coq, ce qui signifie que $P \wedge Q \wedge R$ dénote en fait $P \wedge (Q \wedge R)$. Comme on va le voir ci-dessous, ceci n'a pas tellement d'importance ici. *)

(** ** Disjonction `[V]` *)

(** En coq, le "ou logique", aussi appelée opérateur de disjonction est noté `[V]`.
Une preuve de $[P \vee Q]$ est une preuve de P ou une preuve de Q .
*)

(** *** Prouver une disjonction : `[left]` et `[right]` *)

(** Pour pouvoir prouver une disjonction (par exemple $[P \vee Q]$) en logique

intuitionniste, il faut dire explicitement si l'on donne une preuve de l'opérande de gauche (ici [P]) ou de celui de droite (ici [Q]). Ceci se fait avec les tactiques [left] et [right]. *)

(** *** Utiliser une disjonction : [destruct] *)

(** Utiliser une disjonction (par exemple $P \vee Q$) est un peu plus délicat.

On va alors faire une preuve `_par cas_` :

- Dans un premier temps, on prouve le but sous l'hypothèse que l'opérande de gauche (ici [P]) est prouvée.
- Dans un second temps, on prouve le but sous l'hypothèse que l'opérande de droite (ici [Q]) est prouvée. *)

`destruct H as [HP | HQ].`

(* Noter beaucoup de choses ici :

- la syntaxe avec la barre verticale dans les crochets pour le "ou"
- il y a maintenant deux sous-buts, un par cas. En fait ce ne sont pas les sous-buts qui diffèrent, mais les contextes (hypothèses). *)
- (* cas 1 : hypothèse (HP) *)

(** De la même manière que pour $[\wedge]$, on peut utiliser un "intro pattern" pour détruire une disjonction directement. *)

Theorem `or_comm1'` : forall P Q : Prop, $P \vee Q \rightarrow Q \vee P$.

Proof.

`intros P Q [HP | HQ].`

- `right. assumption.`
- `left. assumption.`

`Qed.`

Négation `[~]` *)

(** La négation en coq (en fait en logique intuitionniste) est aussi un peu particulière par rapport à la négation d'un cours de mathématique usuel. En effet, on ne définit pas la négation d'une proposition par sa table de vérité mais en ajoutant une nouvelle proposition appelée [False].

*)

(** *** La proposition [False] *)

`Check False.`

(** La proposition [False]

désigne le faux, l'absurde, souvent notée en mathématiques $\perp$

(symbole `_bottom_`). Elle est définie par la règle appelée `_ex falso quod libet_` (du faux, on peut déduire n'importe quoi), aussi appelée `_principe d'explosion_`.

*)

(** Si [False] est dans les hypothèses, alors la tactique [destruct] appliquée à celle-ci permet simplement de terminer la preuve. *)

Theorem `ex_falso_quod_libet` : forall P : Prop, $\text{False} \rightarrow P$.

Proof.

(* Soit P une proposition quelconque. *)

`intros P.`

(* Supposons (hypothèse (contra)) que False est prouvée. *)

`intros contra.`

(* Explosion : le faux fait partie des hypothèses. *)

`destruct contra.`

Qed.

(** La tactique [exfalso] remplace le but courant par False.
En effet, si l'on prouve False on prouve tout de toute façon. *)

Theorem stupide : forall P Q : Prop, (P -> False) -> P -> (Q -> P -> Q).

Proof.

(* Soient P et Q deux propositions quelconques. *)

intros P Q.

(* On suppose

(HnP) : P -> False

(HP) : P *)

intros HnP HP.

(* Comme le False implique n'importe quoi, il suffit de prouver False. *)

exfalso.

(* Par HnP il suffit de prouver P. *)

apply HnP.

(* Or P est vrai par hypothèse. *)

assumption.

Qed.

(** *** Réfuter une proposition *)

(** En logique intuitionniste, plutôt que de dire qu'une proposition est
faussee, on dit qu'elle est _contradictoire_, ce qui signifie qu'ajouter
cette proposition dans les hypothèse mène à une explosion : une preuve de
[False] (et donc, par _ex falso quod libet_ de _toutes_ les propositions).

La _négation_ d'une proposition P, notée en coq [$\sim$ P], ou encore [not P],
est en fait simplement une _notation_ pour [P -> False].

*)

(** On peut _déplier_ une notation, c'est-à-dire la remplacer par ce qu'elle
représente avec la tactique [unfold] (littéralement "déplier"). *)

Theorem exemples_unfold : forall A B C : Prop, $\sim$ A -> $\sim$ B -> $\sim$ C -> $\sim$ A $\wedge$ $\sim$ C.

Proof.

intros A B C HnA HnB HnC.

(* On peut préciser dans quel hypothèse on souhaite déplier. *)

unfold not in HnA.

(* Éventuellement, dans plusieurs hypothèses d'un coup. *)

unfold not in HnB, HnC.

(* Si on ne précise pas, c'est dans le but. *)

unfold not.

(* fin de la preuve de ce théorème idiot : *)

split.

- assumption.

- assumption.

Qed.

(** En mathématique, la _contraposée_ de l'implication [A -> C] est
[$\sim$ C -> $\sim$ A]. On vient de voir que [A -> C] implique [$\sim$ C -> $\sim$ A]. En
logique classique (voir plus loin) la réciproque est vraie : une
implication et sa contraposée sont équivalentes. *)

(** *** La double négation *)

(** En logique classique, pour toute proposition A, $\sim\sim$ A est

équivalente à A. En logique intuitionniste, ce n'est pas toujours le cas. $\sim A$ signifiant "A est contradictoire", c'est-à-dire "A fait tout exploser", $\sim\sim A$ signifie "Il est contradictoire d'affirmer que A est contradictoire", autrement dit "A est non contradictoire", ou encore "A ne fait pas tout exploser".

La différence entre A et $\sim\sim A$ est semblable à la différence entre "Je dis que c'est vrai" et "J'ai pas dit que c'était faux". On peut décider par convention que les deux phrases veulent dire la même chose (ie se placer dans une logique classique), mais ce n'est pas obligatoire.

$\sim\sim A$ est une notation pour $(A \rightarrow \text{False}) \rightarrow \text{False}$. *)

(** ** Équivalence [$\leftrightarrow$] *)

Locate " $\leftrightarrow$ ".

Print iff.

(** L'équivalence logique est notée en coq [$\leftrightarrow$].

Une preuve de $[P \leftrightarrow Q]$ est à la fois une preuve de $[P \rightarrow Q]$ et une preuve de $[Q \rightarrow P]$. Les tactiques [destruct] (pour utiliser une équivalence) et [split] (pour prouver une équivalence) s'appliquent de la même manière que pour la conjonction.

*)

Theorem iff_sym : forall P Q : Prop, $P \leftrightarrow Q \rightarrow Q \leftrightarrow P$.

Proof.

(* Soient P et Q deux propositions quelconques. *)

intros P Q.

(* On suppose (H) : $P \leftrightarrow Q$. *)

intros H.

(* De (H) on déduit :

(HPQ) : $P \rightarrow Q$

(HQP) : $Q \rightarrow P$ *)

destruct H as [HPQ HQP].

(* Pour prouver $P \leftrightarrow Q$, il suffit de prouver $P \rightarrow Q$ et $Q \rightarrow P$. *)

split.

- (* Preuve de $P \rightarrow Q$: *)

assumption.

- (* Preuve de $Q \rightarrow P$: *)

assumption.

Qed.

(** ** Appliquer un théorème. *)

(** En coq, on peut fournir des _arguments_ à un théorème pour produire des formules déjà prouvées. *)

Module ThmPlayground.

(* Module pour ne pas polluer le reste du fichier avec les paramètres A, B et C. *)

Parameters A B C : Prop.

Check and_comm1.

(** On peut _appliquer_ le théorème and_comm1 à deux propositions. *)

Check and_comm1 A B.

(** Ou bien à une seule, l'autre restant universellement quantifiée. *)

Check and_comm1 A.

(** On peut même donner en argument des propositions plus compliquées. *)

Check and_comm1 (A $\rightarrow$ B) (B $\rightarrow$ C).

End ThmPlayground.

(** On peut alors utiliser dans des preuves des théorèmes déjà prouvés. *)
 Theorem and_comm : forall P Q : Prop, (P ∧ Q) <-> (Q ∧ P).
 Proof.
 intros P Q.
 split.
 - apply (and_comm1 P Q).
 - apply (and_comm1 Q P).
 Qed.

TP2

(** ** Définition inductive des entiers naturels *)

(* Le type nat est un type inductif: il est fait de constantes et de constructeurs, qui peuvent être recursifs, c'est à dire faire référence au type lui-même. *)

Print nat.
 Check O.
 Check S.

(** Pour coq, un entier naturel est soit
 - O (la lettre O qui représente l'entier 0),
 - soit l'image par la fonction S (pour "successeur") d'un entier naturel.

Ainsi :

* O représente zéro
 * S O est le successeur de zéro donc un
 * S (S O) est le successeur de un donc deux
 * ...

Heureusement coq permet la notation usuelle (décimale) de ces entiers, mais ce n'est qu'une notation pratique ! *)

(** Pour coq, un entier naturel est soit
 - O (la lettre O qui représente l'entier 0),
 - soit l'image par la fonction S (pour "successeur") d'un entier naturel.

Ainsi :

* O représente zéro
 * S O est le successeur de zéro donc un
 * S (S O) est le successeur de un donc deux
 * ...

Heureusement coq permet la notation usuelle (décimale) de ces entiers, mais ce n'est qu'une notation pratique ! *)

(** On dit que la constante O et la fonction S sont les
 constructeurs du type nat. En coq :

- les résultats de constructeurs différents sont toujours différents,
- pour deux termes composés uniquement de constructeurs, s'ils s'écrivent différemment, alors ils sont différents.

La tactique [discriminate] cherche dans les hypothèse une égalité contradictoire de ce type (des termes qui s'écrivent différemment avec les constructeurs mais sont supposés égaux). *)

(** ** Addition des entiers naturels *)

(** Définissons maintenant l'addition des entiers relatifs. Dans un premier temps, on note $\text{add } n \ m$ la somme de n et m . *)

```
Fixpoint add (n m : nat) : nat :=
  match n with (* "faire correspondre n avec" *)
  0 => m (* Si n = 0, alors n + m = m. *)
  | S p => S (add p m) (* (S p) + m = S (p + m). *)
  end.
```

(** Explications : $\text{add } n \ m$ regarde si n vaut 0, si oui il renvoie m , sinon c'est que $n = S \ p$ pour un certain p .
On calcule alors $(\text{add } p \ m)$ et on prend le successeur de ce nombre là.

Cette définition de l'addition est _récursive_ (en coq, "Fixpoint" pour une définition récursive) : la fonction add est elle-même utilisée dans sa définition. Pourtant il n'y a pas de raisonnement circulaire, cette définition permet bien de calculer (sinon, coq ne nous aurait pas laissé faire de toute façon).

Cette définition _est un programme_ : on a non seulement déclaré une fonction qui prend deux entiers et en retourne un, mais en plus on a dit _comment_ calculer le résultat.

Ceci permet à Coq de faire les calculs lui-même :

*)

(** **** Exercice qui a l'air bête mais est très important :

Calculer à la main (papier et crayon), en utilisant la définition de [add] :

$\text{add } (S \ (S \ O)) \ (S \ (S \ (S \ O)))$.

*)

(** En fait Coq dispose des notations usuelles pour l'addition (et la plupart des opérations usuelles. *)

Compute 2 + 5.

(** Nous allons maintenant devoir prouver des égalités.

- La tactique [simpl] demande à coq de simplifier une expression dans le but en utilisant la définition d'une (ou plusieurs) fonctions, autant que possible.
- La tactique [reflexivity] demande à coq de conclure la preuve en constatant que les membres de gauche et de droite d'une égalité sont les mêmes. *)

(* N'oubliez pas [discriminate] pour réfuter une égalité. *)

(** ** La tactique [rewrite] *)

(** [rewrite] est une tactique très importante. Elle permet de réécrire à l'aide d'une égalité. Si vous avez une hypothèse

($H : a = b$)

alors [rewrite H] va transformer chaque [a] du but par un [b]. *)

(** ** Preuve par cas : destruct (encore) *)

(** La façon dont sont construits les naturels en coq fait apparaître deux _cas_ bien différents : un entier naturel, disons n vaut

- soit 0

- soit (S n') pour n' un autre entier.

En coq, la tactique destruct sur une variable de type entier (en général de n'importe quel type inductif) permet les preuves par cas. *)

Theorem double_eq_0_is_0 : forall n : nat, n + n = 0 -> n = 0.

Proof.

(* Soit n un entier naturel. *)

intros n.

(* On suppose (hyp. (H)) que $n + n = 0$. *)

intros H.

(* Preuve par cas : n est soit nul soit successeur d'un entier n'. *)

destruct n as [|n'].

- (* Premier cas : $n = 0$. *)

reflexivity.

- (* Deuxième cas : $n = S\ n'$ pour un certain $n' : \text{nat}$. *)

(* Nous allons montrer que ce cas est impossible. *)

(* Par définition de +, l'hypothèse H devient $S(n' + S\ n') = 0$. *)

simpl in H.

(* Or, il est impossible que le successeur d'un naturel soit nul, par définition des entiers naturels. *)

discriminate.

Qed.

(** Dans la tactique destruct n as [|n'] on retrouve la syntaxe

[quelque_chose | autre_chose], qu'on avait vue lorsqu'on détruit les V.

Ici :

- quelque_chose est vide car c'est le cas $n = 0$, donc avec le constructeur sans argument

- autre_chose est n' : on nomme l'entier n' tel que $n = (S\ n')$. Bien sûr on aurait pu choisir n'importe quel autre nom que n'. *)

(** Remarque : on vient de voir qu'on peut aussi utiliser simpl dans une hypothèse à l'aide de la syntaxe simpl in ... *)

** ** Preuve par induction (ou récurrence) *)

(** simpl, destruct, reflexivity et rewrite ne suffisent pas toujours : *)

Theorem add_0_r : forall n : nat, n + 0 = n.

Proof.

intros n.

simpl. (* Aïe, simpl ne marche pas *)

(* On peut essayer une preuve par cas : *)

destruct n as [|n'|.

- (* le cas $n = 0$ se passe bien *)

simpl. reflexivity.

- (* mais là on va être bloqué... *)

simpl.

(* Il nous faudrait une preuve que $n' + 0 = 0$...

On est bloqué. *)

Abort. (* La commande Abort abandonne une preuve. *)

(** Nous avons besoin de prouver ce théorème par induction. Une preuve par induction, aussi appelée preuve par récurrence sur entier naturel n a deux parties

- on prouve la proposition dans le cas où n vaut 0,

- puis dans le cas où n s'écrit $S\ n'$ pour un certain n' , on prouve la propriété pour n sous l'hypothèse qu'elle est vraie pour n' (on appelle cette hypothèse "hypothèse d'induction"). *)

Abort. (* La commande Abort abandonne une preuve. *)

(** Nous avons besoin de prouver ce théorème par induction. Une preuve par induction, aussi appelée preuve par récurrence sur entier naturel n a deux parties

- on prouve la proposition dans le cas où n vaut 0,

- puis dans le cas où n s'écrit $S\ n'$ pour un certain n' , on prouve la propriété pour n sous l'hypothèse qu'elle est vraie pour n' (on appelle cette hypothèse "hypothèse d'induction"). *)

Theorem add_0_r : forall (n : nat), n + 0 = n.

Proof.

induction n as [|n' IH].

- (* On suppose que $n = 0$. *)

(* Par définition de $+$, $0 + 0$ vaut 0. *)

simpl.

(* Les deux membres de l'égalité sont les mêmes, ce cas est terminé. *)

reflexivity.

- (* On suppose que $n = S\ n'$ et que n' vérifie

(IH) : $n' + 0 = n'$. *)

(* Par définition de $+$, $S\ n' + 0 = S\ (n' + 0)$. *)

simpl.

(* On remplace, grâce à (IH), le terme $(n' + 0)$ par n' . *)

rewrite IH.

(* Les deux membres de l'égalité sont les mêmes, c'est terminé. *)

reflexivity.

Qed.

(** Dans la tactique induction n as [|n' IH], il y a :

- rien à gauche de la barre | car c'est le cas 0, sans argument

- à droite de la barre | il y a n' car c'est le cas $n = (S\ n')$ et

il faut bien donner un nom à cette nouvelle variable et aussi un nom (ici IH) pour l'hypothèse que n' vérifie la propriété qu'on cherche à prouver. *)

(** Avec les tactiques intros, apply, destruct, simpl, discriminate, exfalso, reflexivity, induction et rewrite vous pouvez écrire

énormément de preuves *)

(* Indice : utiliser les théorèmes précédents avec la tactique `rewrite`. Par exemple, si le but contient une expression du type `Truc + S Bidule`, alors `[rewrite add_succ_r]` remplace cette expression par `S (Truc + Bidule)`. *)

**** ** La multiplication *)**

(La multiplication est aussi définie récursivement : *)**

```
Fixpoint mul (n m : nat) :=  
  match n with  
  | 0 => 0  
  | S p => m + mul p m  
end.
```

(Avec des mots :**

- si $n = 0$, alors $\text{mul } n \ m = 0$.
- sinon, $n = (S \ p)$ pour un certain p , et alors $\text{mul } (S \ p) \ m = m + (\text{mul } p \ m)$.

Réécrite avec les notations usuelles, la dernière égalité est bien compréhensible :

$$(p + 1) * m = p * m + 1 * m = p * m + m = m + p * m.$$

Dans la bibliothèque de `coq`, la notation $(n * m)$ correspond à $(\text{mul } n \ m)$.

*)

`Compute (mul 2 3).`

`Compute (2 * 3).`

(**** Exercice papier-crayon :**

Évaluer à la main `mul 2 3`. Les évaluations de l'addition peuvent être faites directement. *)

(** Le connecteur `[forall]` *)**

(Le connecteur "pour tout" s'écrit `(forall)` en Coq :**

`[forall x : X, P x]`

est la proposition qui dit que pour tout x de type X , la proposition $P \ x$ est vraie. Il a déjà été vu en pratique, et il s'agit ici principalement de révisions.

*)

(*** Premiers exemples**

Pour les exemples à venir, on se donne :

- un type quelconque X
- deux `_prédicats_` P et Q sur X , ce qui revient en `coq` à des fonctions de X vers le type `Prop` des propositions.

*)

(La tactique `[trivial]` essaie de terminer une preuve de façon un peu automatique (par exemple, essaie `[assumption]` et `[reflexivity]`). Entre autres choses, elle sait terminer une preuve de `True`, donc peut être utilisée à la place de `[apply I]` *)**

(Rappel : pour utiliser une proposition commençant par un `forall`,**

on peut utiliser les tactiques [exact] ou [apply].

Le lemme suivant (un peu idiot) s'énoncerait de la façon suivante :
"Soit x_0 un élément du type X . Si P est satisfait par tout élément x du type X , alors P est satisfait par x_0 ." *)

Lemma apply_univ1 ($x_0 : X$) : (forall $x : X$, $P\ x$) -> ($P\ x_0$).

Proof.

(* On suppose (hypothèse H), ($P\ x$) pour tout x de type X .)

intros H .

(* On doit montrer ($P\ x_0$), mais c'est exactement notre hypothèse H appliquée à x_0 .)

exact ($H\ x_0$).

Qed.

(** Même chose avec [apply]. *)

Lemma apply_univ2 ($x_0 : X$) : (forall $x : X$, $P\ x$) -> ($P\ x_0$).

Proof.

intros H .

apply ($H\ x_0$).

Qed.

(** En fait, avec [apply], il n'y a pas besoin de préciser qu'on remplace x par x_0 , l'algorithme d'unification de Coq s'en charge pour nous. *)

Lemma apply_univ3 ($x_0 : X$) : (forall $x : X$, $P\ x$) -> ($P\ x_0$).

Proof.

intros H .

apply H .

Qed.

(** Remarque: Il n'y a aucune différence fondamentale entre *)

Lemma test ($x : X$) : $P\ x$.

Admitted.

(** et *)

Lemma test' : forall ($x : X$), $P\ x$.

Admitted.

(** Les deux ont le même type, ce qui dans le cas des lemmes (ou théorèmes) veut dire que les deux prouvent la même chose *)

Check test.

Check test'.

(** En fait, avec [apply], il n'y a pas besoin de préciser qu'on remplace x par x_0 , l'algorithme d'unification de Coq s'en charge pour nous. *)

(** Le connecteur "il existe" s'écrit [exists] dans des mathématiques en Coq.

Il se lit "il existe" : " $\text{exists } x : X, P\ x$ " est la proposition qui dit

qu'il y a (au moins) un x de type X tel que la proposition $P\ x$ est vraie. *)

(** *** Exemples *)

Parameters ($X : \text{Type}$) ($P\ Q : X \rightarrow \text{Prop}$).

Check (exists x, P x).

(** *** Prouver un "il existe" : la tactique [exists] *)

(** En logique intuitionniste, pour prouver une proposition du type "il existe un élément ayant telle propriété", la seule possibilité est _d'exhiber un tel élément_ (appelé témoin). La tactique [exists], suivie de ce témoin pressenti permet d'éliminer le quantificateur [exists]. *)

Lemma exists_n_p_3_eq_8 : exists n : nat, n + 3 = 8.

Proof.

(* Pour montrer qu'il existe n tel que $n + 3 = 8$, _il suffit_ de montrer que $5 + 3 = 8$ (c'est une intuition géniale qui m'a fait penser à 5). *)

exists 5.

(* Par définition de +, $5 + 3 = 8$. *)

simpl.

(* Les deux membres de l'égalité sont les mêmes, la preuve est terminée. *)

reflexivity.

Qed.

(** Pour un type habité (c'est-à-dire ayant au moins un élément), on a le résultat très intuitif suivant. *)

Lemma forall_imp_exists (x0 : X) : (forall x : X, P x) -> (exists x, P x).

Proof.

(* On suppose (hypothèse (H)) que tout x de type X satisfait P. *)

intros H.

(* On sait qu'il y a un x0 de type X par hypothèse.

Pour prouver le but, il suffit de prouver (P x0) *)

exists x0.

(* Par (H) appliquée à x0, (P x0) est vraie *)

apply (H x0).

Qed.

(** Attention, une erreur très courante en mathématiques (ou même dans la logique de tous les jours) consiste à penser qu'avec un ou quelques exemples (donc des preuves de "il existe"), on montre "pour tout" :

- Le polynôme $X^2 - 4$ a des racines réelles donc tous les polynômes en ont.

- Les entiers impairs 3, 5 et 7 sont premiers donc tous les nombres impairs

le sont.

- Cet étudiant est nul en maths, la preuve, il a eu 3 au dernier contrôle.

*)

(** *** Utiliser un "il existe" : tactique [destruct] *)

(** Pour utiliser une proposition avec exists, on utilise de nouveau la tactique destruct. Noter la syntaxe [as [y H]], où y est la variable pour laquelle la propriété va être vraie, et H la preuve que cette propriété est vraie pour y. *)

Lemma exP_imp_exQ (Q : X -> Prop) :

(forall x, P x -> Q x) -> (exists x, P x) -> (exists x, Q x).

Proof.

(* On suppose :

(H1) : forall x, P x -> Q x

(H2) : exists x, P x. *)

```

intros H1 H2.
(* D'après (H2), il y a un certain y de type X qui vérifie
(H') : P y. *)
destruct H2 as [y H'].
(* Pour prouver qu'il existe x tel que Q x, il suffit de prouver Q y. *)
exists y.
(* D'après (H1), pour prouver Q y, il suffit de prouver P y. *)
apply H1.
(* Or, d'après H', on a bien P y. *)
assumption.
Qed.

```

(** ** Ensembles et fonctions *)

(** *** Ensembles et propriétés *)

Module Ensembles.

(* Dans votre cours, une application est définie via un graphe d'application, qui lui-même est un ensemble. Ici nous prenons le point de vue inverse. L'objet fondamental est la fonction, et un ensemble d'éléments de type A (i.e., un sous-ensemble de A), est lui-même une fonction au type bien particulier. *)

(* Un ensemble est défini directement par la propriété qui le caractérise. Si A est un type, un ensemble est un objet P de type (P : A -> Prop).

Voici quelques exemples sur les entiers naturels.
*)

(* Ci-dessous une définition de parties finies de l'ensemble des entiers naturels notés usuellement {0, 1, 2} et {1, 3, 5}. *)

```

Definition ens1 (n : nat) :=
  match n with 0 => True | 1 => True | 2 => True | _ => False end.

```

```

Definition ens2 (n : nat) :=
  match n with 1 => True | 3 => True | 5 => True | _ => False end.

```

(* L'appartenance à un ensemble est alors très simple. Nul besoin de définir la notation $\in$, on se contente de l'application d'une fonction. Un élément (a : A) appartient à l'ensemble (E : A -> Prop) si (E a) est vraie. *)

(** Le théorème suivant dit que 2 appartient à ens1 mais 42 non. *)

```

Theorem ens1_1 : (ens1 2) ∧ ~(ens1 42).

```

Proof.

split.

- simpl. apply I. (* ou [trivial] *)

- unfold not. intros H. simpl in H. assumption. (* ou [destruct H] *)

Qed.

(* On peut définir par récurrence des ensembles d'entiers plus compliqués, ci-dessous, l'ensemble des entiers pairs. *)

```

Fixpoint pair n : Prop :=
  match n with
  | 0 => True (* 0 est pair *)
  | 1 => False (* 1 n'est pas pair *)
  | S (S n') => pair n' (* (n' + 2) est pair si et seulement si n' est pair *)
  end.
Check pair.

```

(** La propriété [pair] est une propriété sur les entiers naturels.
C'est aussi une définition possible de l'ensemble des entiers pairs.

Comme coq est basée sur la théorie des types,
au lieu de : "partie d'un ensemble"
on parle de : "propriété des éléments d'un type"
Autrement dit, au lieu de manipuler l'ensemble
{ x : nat | x est pair }, on manipule le prédicat [pair] directement. *)

** Nous allons voir comment adapter les opérations usuelles de la théorie des parties d'un ensemble dans notre cadre. *)

(** L'ensemble A tout entier est le prédicat sur A qui est toujours vrai. *)
Definition ensemble_total {A : Type} (a : A) := True.
Check ensemble_total.

(** La partie vide de A est le prédicat sur A qui est toujours faux. *)
Definition ensemble_vide {A : Type} (a : A) := False.
Check ensemble_vide.

(** Si A est un type (A : Type), on remplace (a ∈ A) par (a : A) *)

(** Tactique : la tactique [unfold] est d'une grande aide car remplace un nom par sa définition *)

(** **** Exercice : Prouver les théorèmes suivants. *)
Lemma total (A : Type) (a : A) : ensemble_total a.
Proof.
 unfold ensemble_total.
 (* Remplir la preuve ici *)
Admitted. (* Remplacer cette ligne par Qed. *)

Lemma vide (A : Type) (a : A) : ~ (ensemble_vide a).
Proof.
 (* Remplir la preuve ici *)
Admitted. (* Remplacer cette ligne par Qed. *)

(** Les opérations usuelles sur les ensembles sont donc des opérations logiques. Par exemple, l'union de deux ensembles correspond à la disjonction des propriétés qui les définissent *)

Definition union {A : Type} (E F : A -> Prop) :=
 fun (a : A) => (E a ∨ F a).
Check union.

Theorem exple_union :

```

((union ens1 ens2) 5) ∧ ((union ens1 ens2) 1) ∧ ~ ((union ens1 ens2) 42).
Proof.
  unfold union.
  split.
  - right. simpl. apply I.
  - split.
  + left. (* on a le choix ici *) simpl. apply I.
  + intros [H | H].
    * simpl in H. assumption.
    * simpl in H. assumption.
Qed.

```

(** L'inclusion est l'implication entre les propriétés *)

```

Definition incl {A : Type} (E F : A -> Prop) := forall (a : A), (E a -> F a).
Check incl.

```